COMPUTER NETWORKS

(CSC 258)

(B.Sc. CSIT – II/IV)

Unit-4

Part-I

Vipul Gautam

UNIT4 – NETWORK LAYER

4.1. Introduction and Functions
4.2. IPv4 Addressing & Sub-netting
4.3. Class-full and Classless Addressing
4.4. IPv6 Addressing and its Features
4.5. IPv4 and IPv6 **Datagram Formats**
4.6. Comparison of IPv4 and IPv6 Addressing
4.7. Example Addresses: Unicast, Multicast and Broadcast
4.8. Routing
- 4.8.1. Introduction and Definition
- 4.8.2. Types of Routing (Static vs Dynamic, Unicast vs Multicast, Link State vs Distance Vector, Interior vs Exterior)
- 4.8.3. Path Computation Algorithms: Bellman Ford, Dijkstra's
- 4.8.4. Routing Protocols: RIP, OSPF & BGP

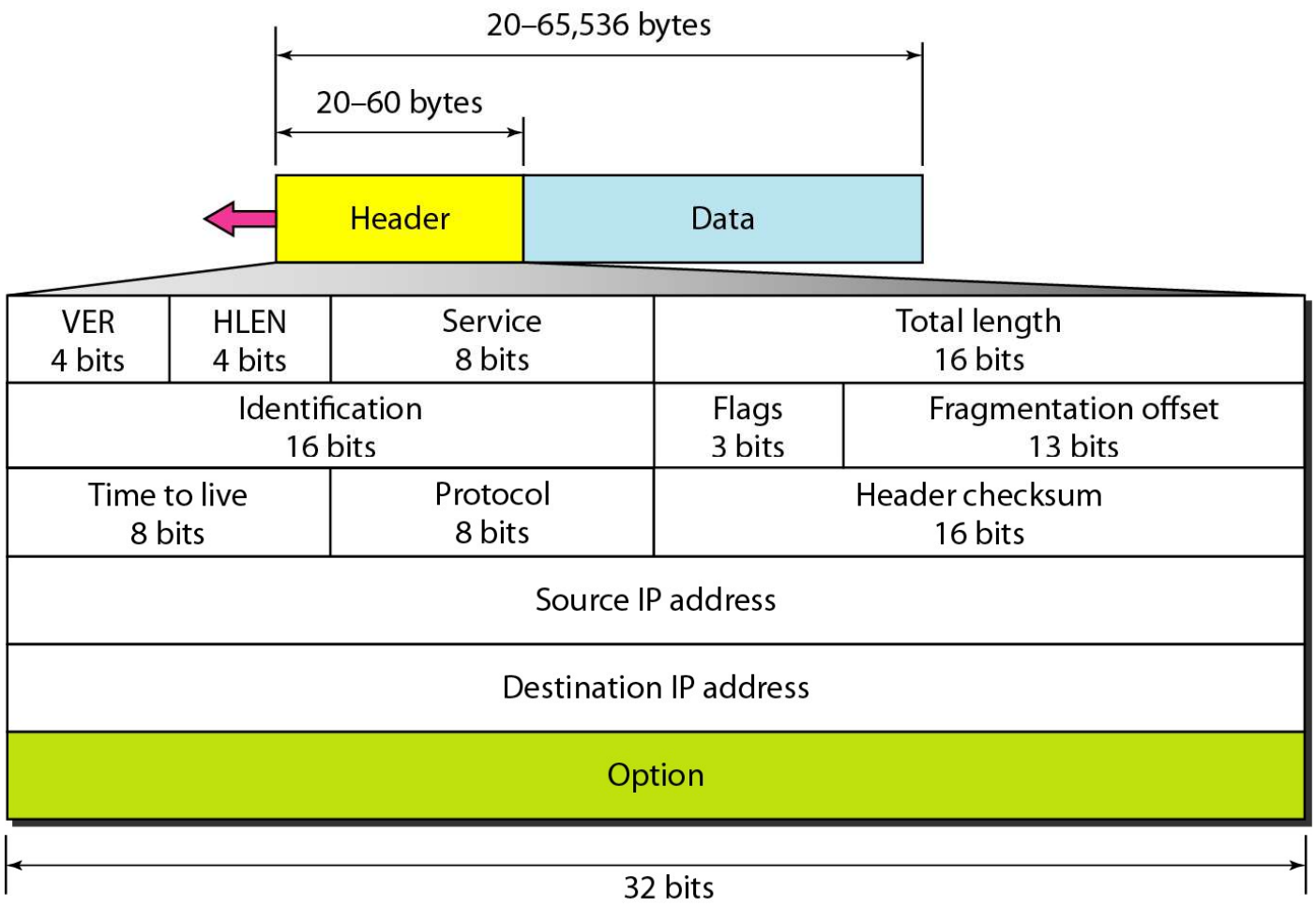
4.9. Overview of IPv4 to IPv6 Transition Mechanisms
4.10. Overview of ICMP/ICMPv6&NATing
4.11. Overview of Network Traffic Analysis
4.12. Security Concepts: Firewall & Router Access Control

IPv4 DATAGRAM

A datagram is a variable-length packet consisting of two parts:
header and **data**.

The header is 20 to 60 bytes in length and contains information essential to routing and delivery.

It is customary in *TCP/IP* to show the header in 4-byte sections.



VERSION (4 bits)

Version (VER):

- This 4-bit field defines the version of the protocol (i.e. 0100).
- This field tells the IPv4 software running in the processing machine that the datagram has the format of version 4.
- All fields must be interpreted as specified in the fourth version of the protocol.
- If the machine is using some other version of IP, the datagram is discarded rather than interpreted incorrectly.

HEADER LENGTH (4 bits)

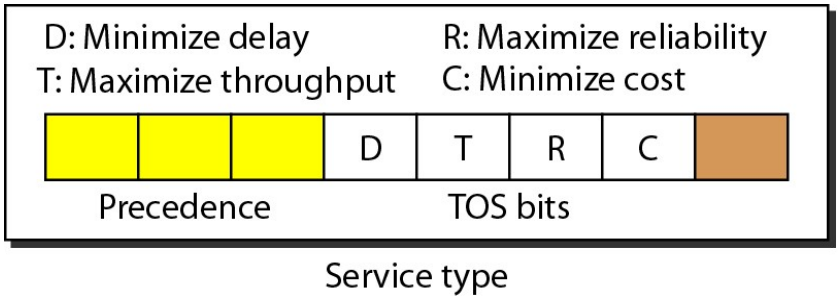
Header length (HLEN):

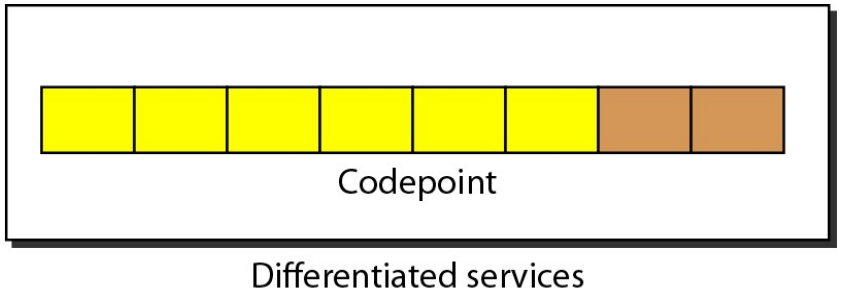
- This 4-bit field defines the total length of the datagram header in 4-byte words.
- This field is needed because the length of the header is variable (between 20 and 60 bytes).
- The field value is multiple of 4 in decimal.
- When there are no options, the header length is 20 bytes, and the value of this field is 0101 which is 5 in decimal. Thus, 5 x 4 = 20.
- When the option field is at its maximum size, the value of this field is 1111 which is 15 in decimal. Thus, 15 x 4 = 60.

SERVICES (8 bits)

Services:

- IETF has changed the interpretation and name of this 8-bit field.
- This field, previously called service type, is now called **differentiated services code point (DSCP)**.





- *The precedence subfield was part of version 4, but never used.*

SERVICE TYPE – PRECEDENCE

1. Service Type: In this interpretation, the first 3 bits are called precedence bits. The next 4 bits are called type of service (TOS) bits, and the last bit is not used.

a. **Precedence** is a 3-bit subfield ranging from 0 to 7 *(000 to 111 in binary)*.

- The precedence defines the priority of the datagram in issues such as congestion.
- If a router is congested and needs to discard some datagrams, those datagrams with lowest precedence are discarded first.
- Some datagrams in the Internet are more important than others.
  - eg, a datagram used for network management

SERVICE TYPE – TOS

b. **TOS (Type of Service)** bits is a 4-bit subfield with each bit having a special meaning.

- Although a bit can be either 0 or 1, one and only one of the bits can have the value of 1 in each datagram.

- With only 1 bit set at a time, we can have five different types of services.

- Application programs can request a specific type of service, depicted in next slide.

TOS Bits	*Description*
0000	Normal (default)
0001	Minimize cost
0010	Maximize reliability
0100	Maximize throughput
1000	Minimize delay

TOS

Examples:

- Interactive activities, activities requiring immediate attention, and activities requiring immediate response need minimum delay.
- Those activities that send bulk data require maximum throughput.
- Management activities need maximum reliability.
- Background activities need minimum cost.

Protocol	*TOS Bits*	*Description*
ICMP	0000	Normal
BOOTP	0000	Normal
NNTP	0001	Minimize cost
IGP	0010	Maximize reliability
SNMP	0010	Maximize reliability
TELNET	1000	Minimize delay
FTP (data)	0100	Maximize throughput
FTP (control)	1000	Minimize delay
TFTP	1000	Minimize delay
SMTP (command)	1000	Minimize delay
SMTP (data)	0100	Maximize throughput
DNS (UDP query)	1000	Minimize delay
DNS (TCP query)	0000	Normal
DNS (zone)	0100	Maximize throughput

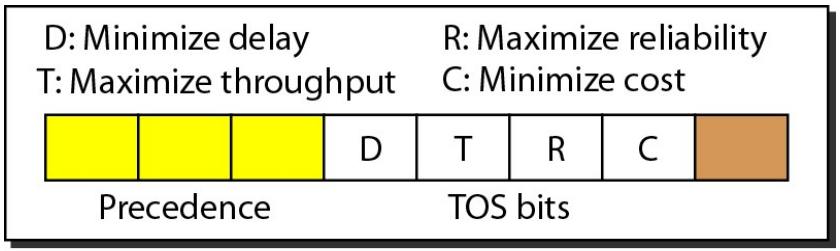
DIFFERENTIATED SERVICES (8 bits)

2. Differentiated Services

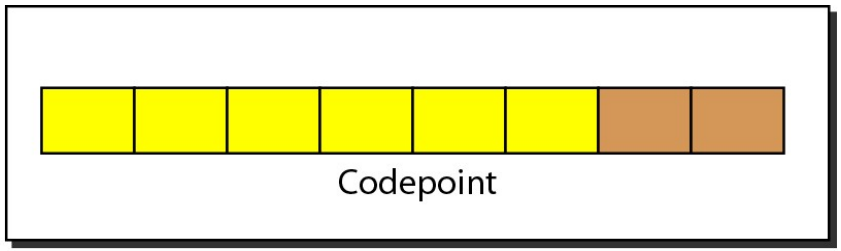
- In this interpretation, the first 6 bits make up the codepoint subfield, and the last 2 bits are normally not used (but can be used for ECN – Explicit Congestion Notification).
- The codepoint subfield can be used in two different ways.

  a. When the 3 rightmost bits are 0s, the 3 leftmost bits are interpreted the same as the precedence bits in the service type interpretation.

  - In other words, it is compatible with the old interpretation.



Service type



Differentiated services

DIFFERENTIATED SERVICES (8 bits)

b. When the 3 rightmost bits are not all 0s, the **6 bits define 64 services** based on the priority assignment by the Internet or local authorities according to given table.

- The **first category** contains **32 service** types; the **second** and the **third** each contain **16**.
- The first category (numbers 0,2,4,6,...,62) is assigned by the Internet authorities (IETF).
- The second category (3, 7, 11, 15,…,63) can be used by local authorities (organizations).
- The third category (1, 5, 9,…,61) is temporary and can be used for experimental purposes.
- Note that the numbers are not contiguous. If they were, the first category would range from 0 to 31, the second from 32 to 47, and the third from 48 to 63.

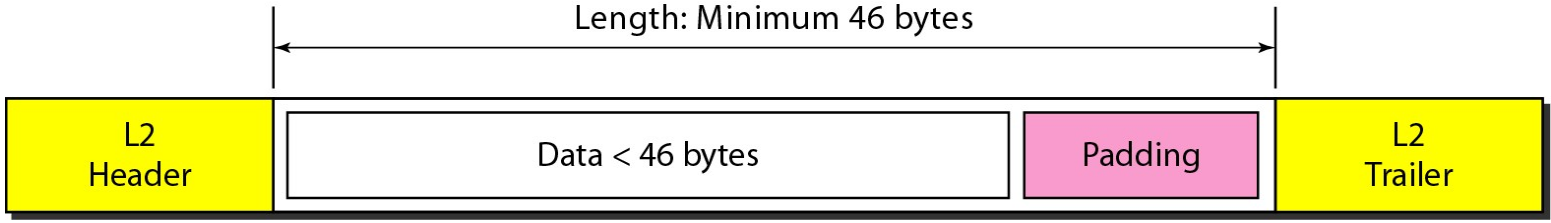
Category	*Codepoint*	*Assigning Authority*
1	XXXXXO	Internet
2	XXXXl1	Local
3	XXXXOI	Temporary or experimental

TOTAL LENGTH (16 bits)

- This is a 16-bit field that defines the total length (header plus data) of the IPv4 datagram in bytes.
- To find the length of the data coming from the upper layer, we need to subtract the header length from the total length.
- Length of data = total length - header length
- The header length can be found by multiplying the value in the HLEN field by 4.
- Since the field length is 16 bits, the total length of the IPv4 datagram is limited to 65,535 (2^{16} - 1) bytes, of which 20 to 60 bytes are the header and the rest is data from the upper layer.
- Though a size of 65,535 bytes might seem large, the size of the IPv4 datagram may increase in the near future as the underlying technologies allow even more throughput (greater bandwidth).
- Some physical networks are not able to encapsulate the datagram of 65,535 bytes in their frames, thus the datagram must be fragmented for those networks.

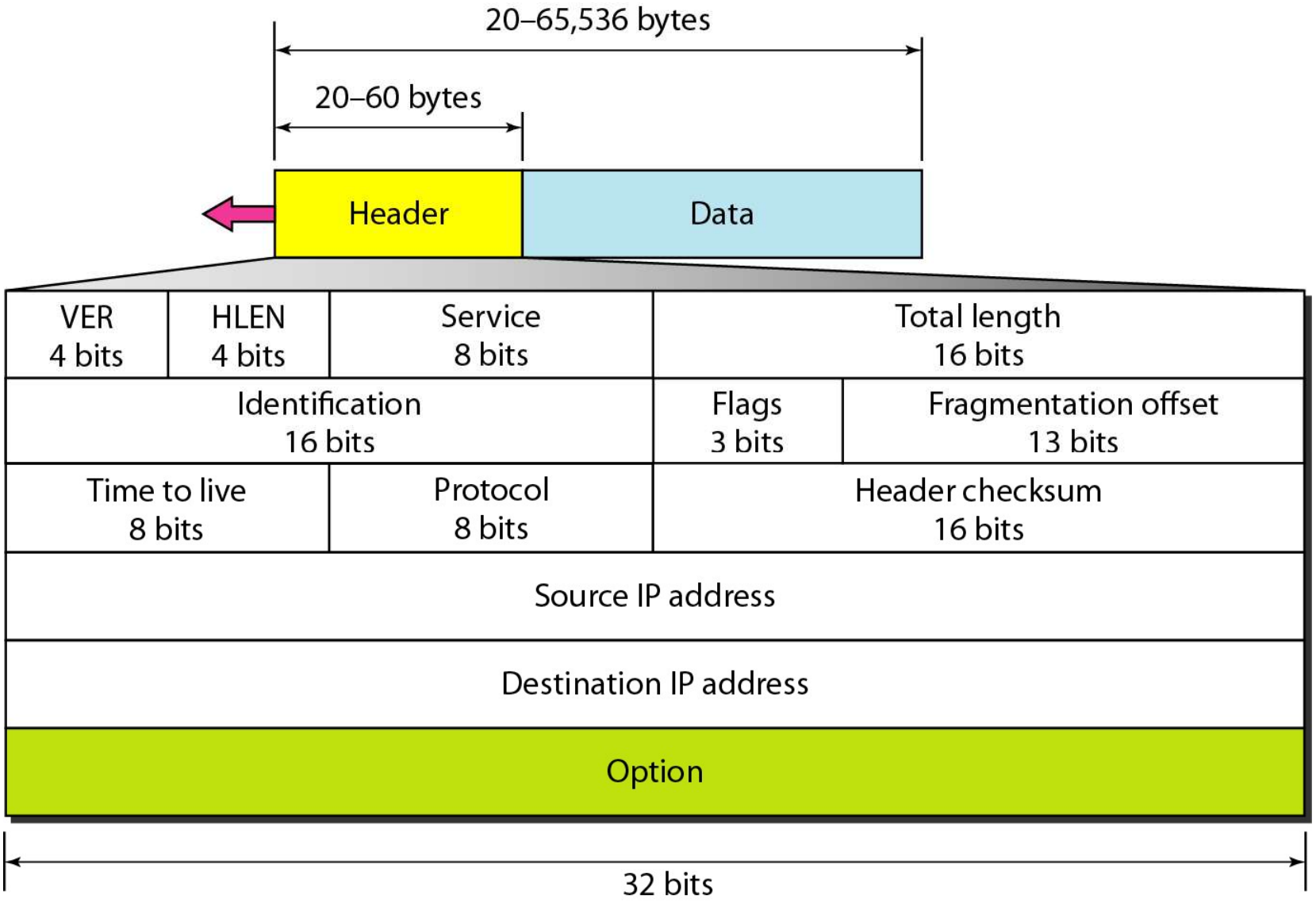
WHY TOTAL LENGTH?

- When a machine (router or host) receives a frame, it drops the header and the trailer, leaving the datagram.
- Why include an extra field that is not needed?
  - The answer is that in many cases we really do not need the value in this field.
- However, there are occasions in which the datagram is not the only thing encapsulated in a frame; it may be that padding has been added.
- For example, the Ethernet protocol has a minimum and maximum restriction on the size of data that can be encapsulated in a frame (46 to 1500 bytes).
- If the size of an IPv4 datagram is less than 46 bytes, some padding will be added to meet this requirement. In this case, when a machine decapsulates the datagram, it needs to **check the total length field** to determine how much is really data and how much is padding



FRAGMENTATION

- **Identification, Flags and Fragmentation Offset** are fields used in fragmentation.
- A datagram can travel through different networks.
- Each router decapsulates the IPv4 datagram from the frame it receives, processes it, and then encapsulates it in another frame.
- The format and size of the received frame depend on the protocol used by the physical network through which the frame has just traveled.
- The format and size of the sent frame depend on the protocol used by the physical network through which the frame is going to travel.
- For example, if a router connects a LAN to a WAN, it receives a frame in the LAN format and sends a frame in the WAN format.

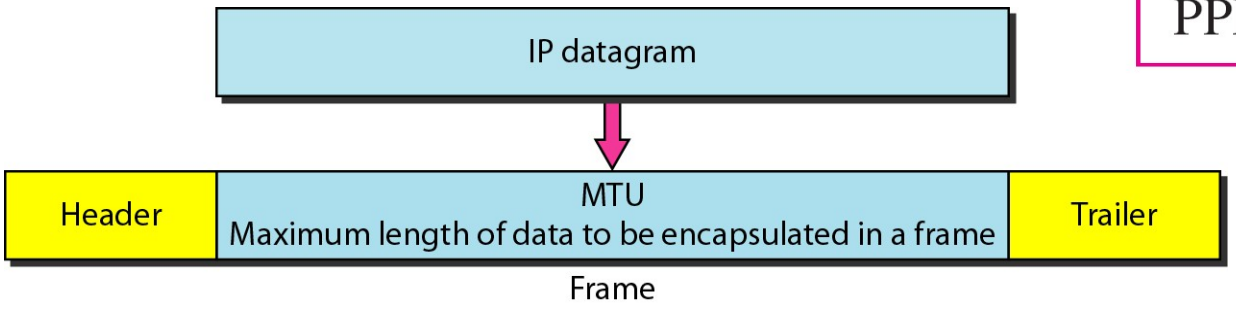

20–65,536 bytes
20–60 bytes
Header
Data
VER
4 bits
HLEN
4 bits
Service
8 bits
Total length
16 bits
Identification
16 bits
Flags
3 bits
Fragmentation offset
13 bits
Time to live
8 bits
Protocol
8 bits
Header checksum
16 bits
Source IP address
Destination IP address
Option
32 bits

MAXIMUM TRANSFER UNIT (MTU)

- Each data link layer protocol has its own frame format in most protocols.
- One of the fields defined in the format is the maximum size of the data field.
- In other words, when a datagram is encapsulated in a frame, the total size of the datagram must be less than this maximum size, which is defined by the restrictions imposed by the hardware and software used in the network (figure in next slide).
- The value of the MTU depends on the physical network protocol.
- Figure in next slide shows the values for some protocols.

MTU & PROTOCOL

Protocol	*MTU*
Hyperchannel	65,535
Token Ring (16 Mbps)	17,914
Token Ring (4 Mbps)	4,464
FDDI	4,352
Ethernet	1,500
X.25	576
PPP	296



MTU are in Bytes

FRAGMENTATION

- To make the IPv4 protocol independent of the physical network, the designers decided to make the maximum length of the IPv4 datagram equal to 65,535 bytes.
- This makes transmission more efficient if we use a protocol with an MTU of this size.
- However, for other physical networks, we must divide the datagram to make it possible to pass through these networks. This is called fragmentation.
- The source usually does not fragment the IP packet (earlier networks).
- The transport layer will instead segment the data into a size that can be accommodated by IP and the data link layer in use.
- If a datagram is fragmented at Network Layer, each fragment has its own header with most of the fields repeated, but with some changed.
- A fragmented datagram may itself be fragmented if it encounters a network with an even smaller **MTU**. In other words, a datagram can be fragmented several times before it reaches the destination.

FRAGMENTATION

- In recent networks, a datagram can be fragmented by the source host or any router in the path although there is a tendency to limit fragmentation only at the source.
- ***PMTU*** – Path MTU Discovery from source to destination helps Source to identify the least MTU size, accordingly fragments are made by the Source.
- The reassembly of the datagram, however, is done only by the destination host because each fragment becomes an independent datagram.
- The fragmented datagram can travel through different routes, and we can never control or guarantee which route a fragmented datagram may take.
- As, all the fragments belonging to the same datagram finally arrive at the destination host thus it is logical to do the reassembly at the final destination.
- An even stronger objection to reassembling packets during the transmission is the loss of efficiency it incurs.

FRAGMENTATION

- When a datagram is fragmented, required parts of the header must be copied by all fragments.
- But the **option** field may or may not be copied.
- The host or router that fragments a datagram must **change the values** of three fields:
  - **flags,**
  - **fragmentation offset, and**
  - **total length.**
- The rest of the fields must be copied.
- And the value of the **checksum** must be **recalculated** regardless of fragmentation.
- The fields that are related to fragmentation and reassembly of an IPv4 datagram are the **identification, flags, and fragmentation offset**.

IDENTIFICATION (16 bits)

- It identifies a datagram originating from the source host.
- The combination of the identification and source IPv4 address uniquely defines a datagram as it leaves the source host.
- To guarantee uniqueness, the IPv4 protocol uses a **counter** to label the datagrams which is initialized to **a positive number**.
- When the IPv4 protocol sends a datagram, it copies the current value of the counter to the identification field and increments the counter by 1.
- As long as the counter is kept in the main memory, uniqueness is guaranteed.
- When a datagram is fragmented, the value in the identification field is copied to all fragments; i.e. **all fragments** have the **same identification number**, the same as the original datagram.
- All the fragments with same identification number are reassembled to the single datagram.

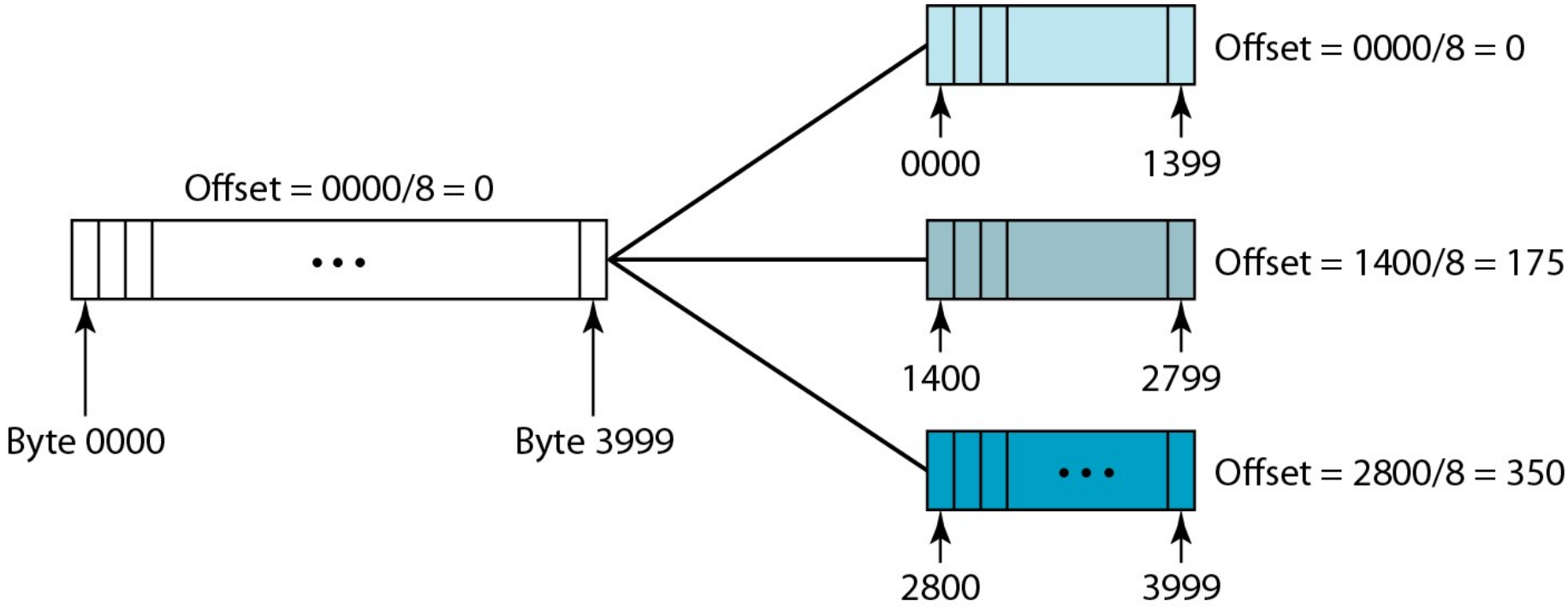
FLAGS (3 bits)

- The first bit is reserved.
- The second bit is called the **do not fragment** bit.
  - If its value is 1, the machine must not fragment the datagram.
    - If it cannot pass the datagram through any available physical network, it discards the datagram and sends an ICMP error message to the source host.
  - If its value is 0, the datagram can be fragmented if necessary.
- The third bit is called the **more fragment** bit.
  - If its value is 1, it means the datagram is not the last fragment; there are more fragments after this one.
  - If its value is 0, it means this is the last or only fragment.

FRAGMENTATION OFFSET (13 bits)

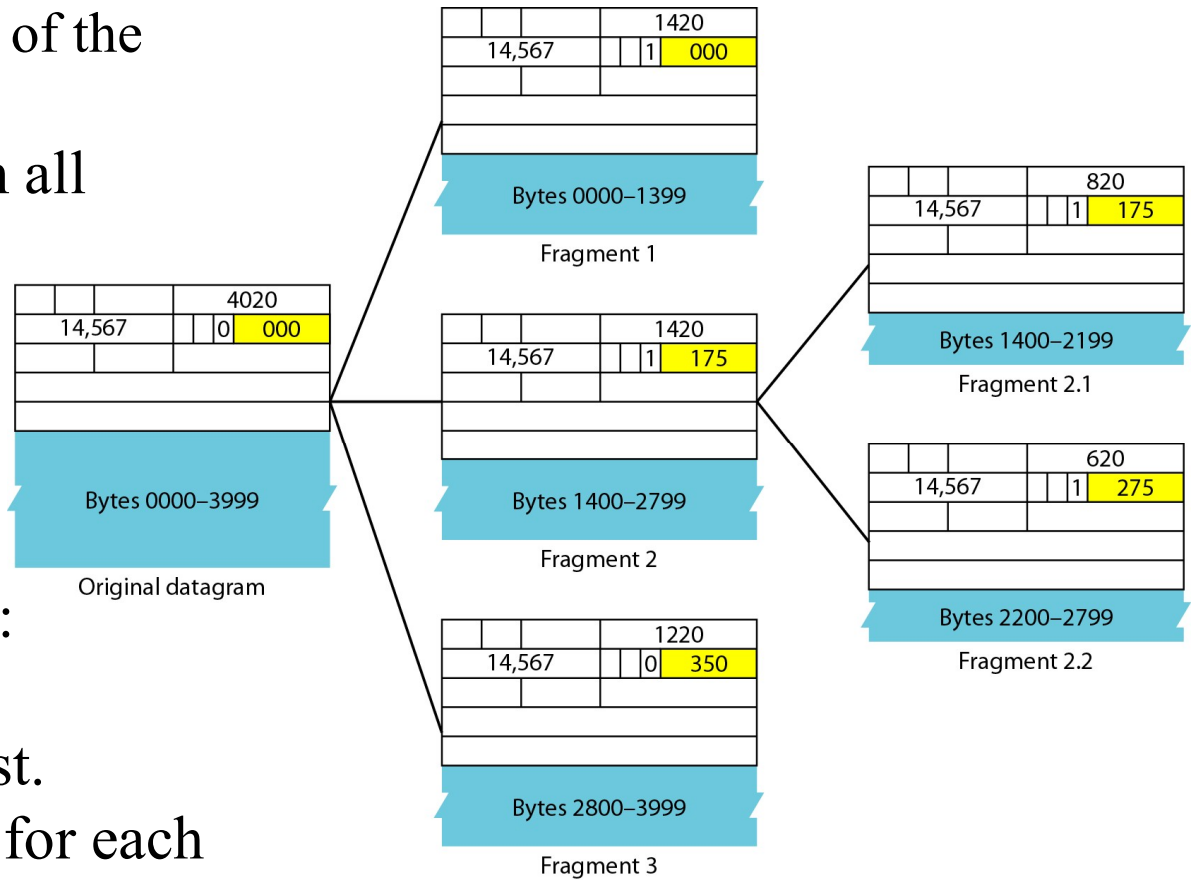
- It shows the relative position of this fragment with respect to the whole datagram.
- It is the offset of the data in the original datagram measured in **units of 8 bytes**.
- Figure in next slide shows a datagram with a data size of 4000 bytes fragmented into **3 fragments**.
- The value of the **fragmentation offset is measured in units of 8 bytes**.
- This is done because the length of the **offset field is only 13 bits** and **cannot represent a sequence of bytes greater than 2^{13}=8192**.
- This forces hosts or routers that fragment datagrams to choose a fragment size so that the first byte number is divisible by 8.
- The bytes in the original datagram are numbered 0 to 3999.
- The first fragment carries bytes 0 to 1399.
  - The offset for this datagram is 0/8 = 0.
- The second fragment carries bytes 1400 to 2799.
  - The offset value for this fragment is 1400/8 = 175.
- Finally, the third fragment carries bytes 2800 to 3999.
  - The offset value for this fragment is 2800/8 =350.

FRAGMENTATION – OFFSET VALUE



FRAGMENTATION – DETAIL VIEW

- The figure shows an expanded view of the fragments of previous example.
- **Identification** field is the same in all fragments.



- Notice the value of the flags field:
  - *more fragments* bit set for all fragments except the last.
- Also, the value of the **offset** field for each fragment is shown in the figure.

FRAGMENTATION – DETAILS

- The above figure also shows what happens if a fragment itself is fragmented.
- In this case the value of the offset field is always relative to the original datagram.
- For example, in the figure, the second fragment is itself fragmented later to two fragments of 800 bytes and 600 bytes, but the offset shows the relative position of the fragments to the original data.
- It is obvious that even if each fragment follows a different path and arrives out of order, the destination host can reassemble the original datagram from the fragments received (if none of them is lost) by using the following strategy:

  1. The first fragment has an offset field value of zero.
  2. Divide the length of the first fragment by 8. The second fragment has an offset value equal to that result.
  3. Divide the total length of the first and second fragments by 8. The third fragment has an offset value equal to that result.
  4. Continue the process. The last fragment has a *more fragment* bit value of 0.

TIME TO LIVE (8 bits)

- A datagram has a limited lifetime in its travel through an internet.
- This field was originally designed to hold a timestamp, which was decremented by each visited router.
- The datagram was discarded when the value became zero.
- But, for this scheme, all the machines should have synchronized clocks and must know how long it takes for a datagram to go from one machine to another.
- Today, this field is used mostly to control the maximum number of hops (routers) visited by the datagram.
- When a source host sends the datagram, it stores a number in this field.
- This value is approximately 2 times the maximum number of routes between any two hosts.
- Each router that processes the datagram decrements this number by 1.
- If this value, after being decremented, is zero, the router discards the datagram.

WHY TTL?

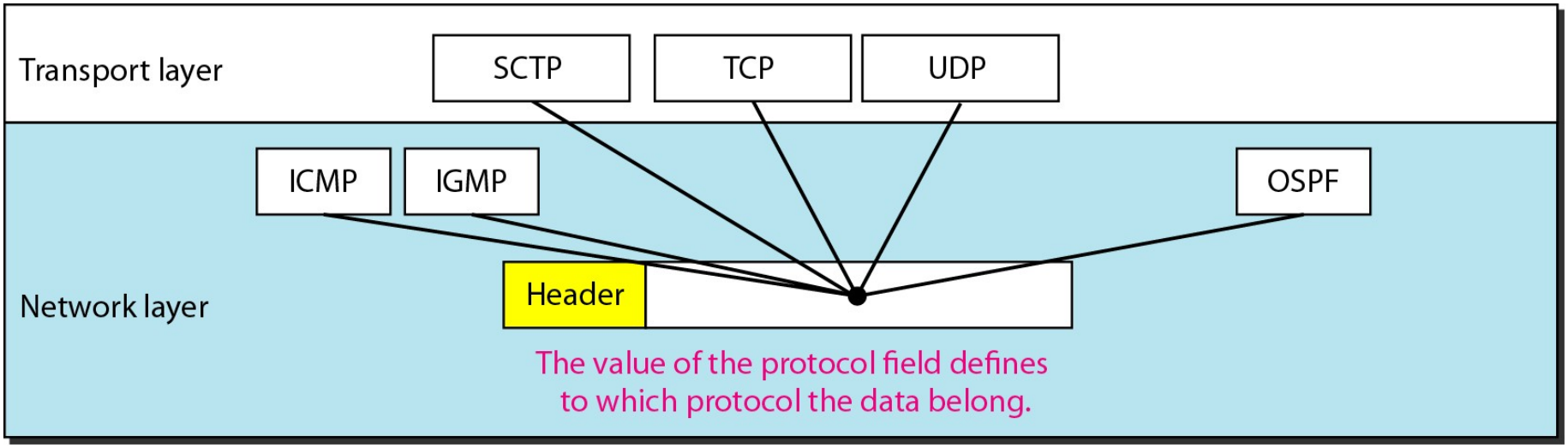
- This field is needed because routing tables on the Internet can become corrupted.
- A datagram may travel between two or more routers for a long time without ever getting delivered to the destination host.
- This field limits the lifetime of a datagram.
- Another use of this field is to intentionally limit the journey of the packet.
  - e.g., if the source wants to confine the packet to the local network, it can store 1 in this field.
  - When the packet arrives at the first router, this value is decremented to 0, and the datagram is discarded.

PROTOCOL (8 bits)

- This 8-bit field defines the higher-level protocol that uses the services of the IPv4 layer.
- An IPv4 datagram can encapsulate data from several higher-level protocols such as TCP, UDP, ICMP, and IGMP.
- This field specifies the final destination protocol to which the IPv4 datagram is delivered.
- In other words, since the IPv4 protocol carries data from different other protocols, the value of this field helps the receiving network layer know to which protocol the data belong.

PROTOCOLS BEING CARRIED

Value	*Protocol*
1	ICMP
2	IGMP
6	TCP
17	UDP
89	OSPF



CHECKSUM [HEADER CHEKSUM]

- The implementation of the checksum in the IPv4 packet follows the same principles as we discussed in Unit 3.
- First, the value of the checksum field is set to 0.
- Then the entire header is divided into 16-bit sections and added together.
- The result (sum) is complemented and inserted into the checksum field.
- The **checksum in the IPv4 packet covers only the header**, not the data.
- There are two good reasons for this.
  - First, all higher-level protocols that encapsulate data in the IPv4 datagram have a checksum field that covers the whole packet. Therefore, the checksum for the IPv4 datagram does not have to check the encapsulated data.
  - Second, the header of the IPv4 packet changes with each visited router, but the data do not. So the checksum includes only the part that has changed.
- **If the data were included for checksum at Network Layer, each router must recalculate the checksum for the whole packet, which means an increase in processing time.**

SOURCE & DESTINATION ADDRESS

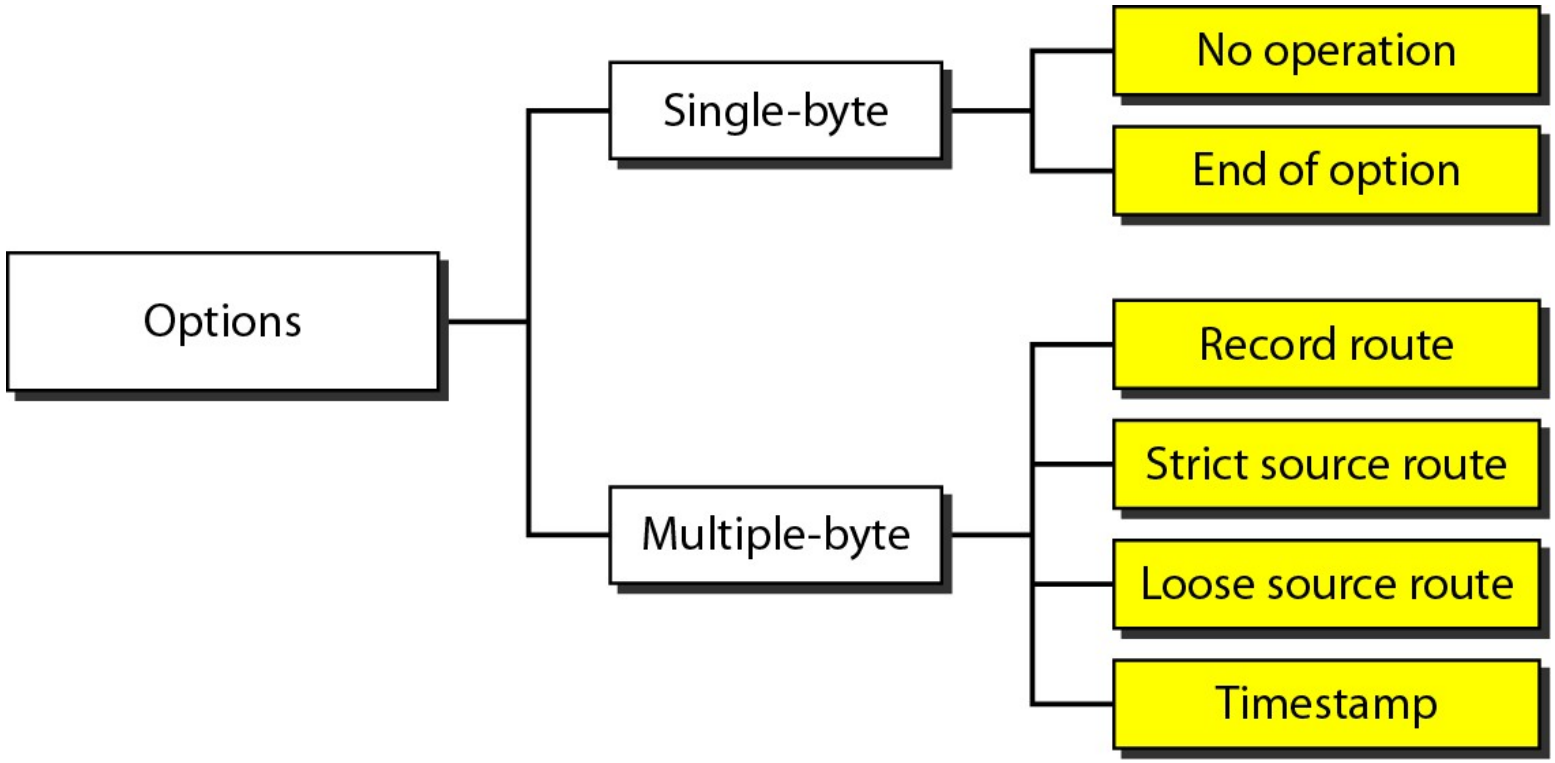
Source address:

- This 32-bit field defines the IPv4 address of the source.
- This field must remain unchanged during the time the IPv4 datagram travels from the source host to the destination host.

Destination address:

- This 32-bit field defines the IPv4 address of the destination.
- This field must remain unchanged during the time the IPv4 datagram travels from the source host to the destination host.

OPTIONS


Options
Single-byte
Multiple-byte
No operation
End of option
Record route
Strict source route
Loose source route
Timestamp

OPTIONS

- The header of the IPv4 datagram is made of two parts:
  - a fixed part and
  - a variable part
- The fixed part is 20 bytes long and was discussed in the previous section.
- The variable part comprises the options that can be a maximum of 40 bytes.
- Options, as the name implies, are not required for a datagram.
- They can be used for network testing and debugging.
- Although options are not a required part of the IPv4 header, option processing is required of the IPv4 software.
- This means that all implementations must be able to handle options if they are present in the header.

OPTIONS

No Operation – It is a **1-byte option** used as a **filler between options**.

End of Option

- An “end-of-option” option is a 1-byte option used for **padding at the end of the option field**.
- It, however, can only be used as the last option.

Record Route

- A record route option is used to **record the Internet routers** that handle the datagram.
- It can list up to nine router addresses.
- It can be used for debugging and management purposes.

OPTIONS

Strict Source Route:

- A strict source route option is used by the source to **predetermine a route** for the datagram as it travels through the Internet.
- Dictation of a route by the source can be useful for several purposes.
- The sender can choose a route with a specific type of service, such as minimum delay or maximum throughput.
- Alternatively, it may choose a route that is safer or more reliable for the sender's purpose. For example, a sender can choose a route so that its datagram does not travel through a competitor's network.
- If a datagram specifies a strict source route, **all the routers defined in the option must be visited** by the datagram.
- A router must not be visited if its IPv4 address is not listed in the datagram. If the datagram visits a router that is not on the list, the datagram is discarded and an error message is issued. If the datagram arrives at the destination and some of the entries were not visited, it will also be discarded and an error message issued.

OPTIONS

Loose Source Route

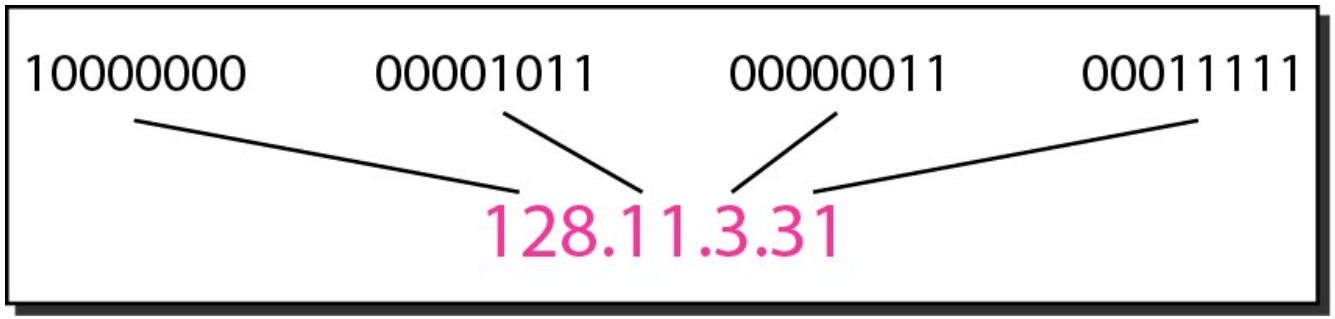
- A loose source route option is similar to the strict source route, but it is less rigid.
- **Each router in the list must be visited**, but the datagram **can visit other routers** also.

Timestamp

- A timestamp option is used to record the time of datagram processing by a router.
- The time is expressed in milliseconds from midnight, Universal time or Greenwich mean time.
- Knowing the time, a datagram is processed can help users and managers track the behavior of the routers in the Internet.
- We can estimate the time it takes for a datagram to go from one router to another.
- We say estimate because, although all routers may use Universal time, their local clocks may not be synchronized.

IPv4 ADDRESS

- An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a device (for example, a computer or a router) to the Internet.
- The address space of IPv4 is $\mathbf{2^{32}}$ or 4,294,967,296.
- Four Octets: 128.11.3.31[Dotted Decimal Representation]
- Each Octet: **8 bits** in binary [0 to 255 in decimal]



IPv4 ADDRESS CLASSES (RFC 791)

Class A 0.0.0.0 → 127.255.255.255 [SM – 255.0.0.0]

Class B 128.0.0.0 → 191.255.255.255 [SM – 255.255.0.0]

Class C 192.0.0.0 → 223.255.255.255 [SM – 255.255.255.0]

Class D 224.0.0.0 → 239.255.255.255 [Multicast]

Class E 240.0.0.0 → 255.255.255.255 [RnD]

Private IP Addresses (RFC 1918):

A 10.0.0.0 → 10.255.255.255 -------/8

B 172.16.0.0 → 172.31.255.255 --------/12

C 192.168.0.0 → 192.168.255.255 ----- /16

IPv4 ADDRESS CLASSES & BLOCK SIZES

	First byte	Second byte	Third byte	Fourth byte
Class A	0			
Class B	10			
Class C	110			
Class D	1110			
Class E	1111			

a. Binary notation

	First byte	Second byte	Third byte	Fourth byte
Class A	0–127			
Class B	128–191			
Class C	192–223			
Class D	224–239			
Class E	240–255			

b. Dotted-decimal notation

Class	*Number of Blocks*	*Block Size*	*Application*
A	$2^7 =$ 128	16,777,216	Unicast
B	$2^{14} =$ 16,384	65,536	Unicast
C	$2^{21} =$ 2,097,152	256	Unicast
D	1	268,435,456	Multicast
E	1	268,435,456	Reserved

FIND CLASSES?

Find the class of each address:

a. 00000001 00001011 00001011 11101111

b. 11000001 10000011 00011011 11111111

c. 14.23.120.8

d. 252.5.15.111

Solution

a. The first bit is 0. This is a class A address.

b. The first 2 bits are 1; the third bit is 0. This is a class C address.

c. The first byte is 14; the class is A.

d. The first byte is 252; the class is E.

DECIMAL TO BINARY

Change the following IPv4 addresses from dotted-decimal notation to binary notation.

a. 111.56.45.78

b. 221.34.7.82

Solution

a. 01101111 00111000 00101101 01001110

b. 11011101 00100010 00000111 01010010

SUBNET MASK

- Subnet Mask help identify Network Portion and Host Portion
- **Subnet Mask when ANDED with IP Address gives the Network Address**
- eg.
  - IP Address: 120.4.5.6 01111000.00000100.00000101.00000110
  - Subnet Mask: 255.0.0.0 11111111.00000000.00000000.00000000
  - Network Address: 120.0.0.0 01111000.00000000.00000000.00000000

Class	*Binary*	*Dotted-Decimal*	*CIDR*
A	**11111111** 00000000 00000000 00000000	**255**.0.0.0	/8
B	**11111111 11111111** 00000000 00000000	**255.255**.0.0	/16
C	**11111111 11111111 11111111** 00000000	**255.255.255**.0	/24

CLASSLESS ADDRESSING

- In a classful addressing, a large part of the available addresses get wasted.
- Classful addressing, which is almost obsolete, is replaced with classless addressing.
- In IPv4 addressing, a block of addresses can be defined as x.y.z.t/**n.**
- Here, x.y.z.t defines one of the addresses and the /**n** defines **number of 1** in **Subnet Mask** called as **Classless Inter Domain Routing (CIDR)** length.

  eg. 192.168.10.11/27

  Subnet Mask (SM) – 255.255.255.224

  Above SM in Binary – 11111111.11111111.11111111.11100000

WHY SUBNETTING?

- Making several subnetworks from a single network (logical subdivision).
- Dividing a larger network into smaller ones for the security and better performance.
  - Different subnetworks can't communicate directly with each other without the presence of router.
  - A large broadcast domain is split to smaller ones, so less congestion occurs.
- IP segregating for different units in an organization for better security.

  e.g., Marketing Dept., Manufacturing Dept., Sales Dept., Storage Dept., Packaging Dept., HR Dept., Finance Dept., IT Dept., Business Development Dept., Monitoring & Supervision Dept., Administration Dept., Management Team., Board of Directors, etc.

Subnetting Class C IP Address into 2 Blocks

- Any one Class C Private Network can have total 256 IP Addresses.
  - e.g. Range of IPs Addresses: 192.168.9.0 to 192.168.9.255
- Network Address: 192.168.9.0 → **11000000.10101000.00001001.00000000**
- Broadcast Address: 192.168.9.255 → **11000000.10101000.00001001.11111111**
- Subnet Mask (Class C): 255.255.255.0→ **11111111. 11111111. 11111111.00000000**
- Now, we need to split this network into two halves:
  - i.e. Total 256 IPs become 128 IPs in one network and 128 IPs in the other.
- 192.168.9.0 to 192.168.9.127 → **0**0000000 to **0**1111111
- 192.168.9.128 to 192.168.9.255 → **1**0000000 to **1**1111111
- Verify the First Changing Bit from Most Significant Bit in Last Octet (Rightmost)
  - We add the changing bit Value along with Subnet Mask of Class C.
- Thus, Subnet Mask becomes → 255.255.255.**1**0000000 = 255.255.255.128

Subnetting Class C IP Address into 4 Blocks

Initially,

- Network Address: 192.168.9.0 → **11000000.10101000.00001001.00000000**
- Broadcast Address: 192.168.9.255 → **11000000.10101000.00001001.11111111**
- Subnet Mask (Class C): 255.255.255.0→ **11111111. 11111111. 11111111.00000000**
- Now, we need to split this network into 4 equal halves:
  - i.e. Total 256 IPs become 64 IPs in each four networks (64 x 4 = 256).
- 192.168.9.0 to 192.168.9.63 → **0**0000000 to **00**111111
- 192.168.9.64 to 192.168.9.127 → **01**000000 to **01**111111
- 192.168.9.128 to 192.168.9.191 → **1**0000000 to **10**111111
- 192.168.9.192 to 192.168.9.255 → **11**000000 to **11**111111
- Verify the Changing Bits from Most Significant Bit in Last Octet (Rightmost Octet)
  - We add the changing bits values along with Subnet Mask of Class C.
- Thus, Subnet Mask becomes → 255.255.255.**11**000000 = 255.255.255.192

FIRST ADDRESS IN THE BLOCK

- A block of IP addresses is granted by an ISP to an organization (customer of ISP) where one of the addresses in the block is 205.16.37.39/**28**.
- The first address in the block can be found by **setting the rightmost "32−n" bits to all 0s**.
- What is the first address in above block?
  - The binary representation of the given address is:

    11001101 00010000 00100101 00100111
  - If we set "32−28=4" rightmost 4 bits to 0, we get

    11001101 00010000 00100101 00100000

First IP in Dotted Decimal → 205.16.37.32

LAST ADDRESS IN THE BLOCK

- The last address in the block can be found by setting the rightmost "32 − **n**" bits to all 1s.
- Example - 205.16.37.39/**28**
- The binary representation of the given address is

  11001101 00010000 00100101 00100111

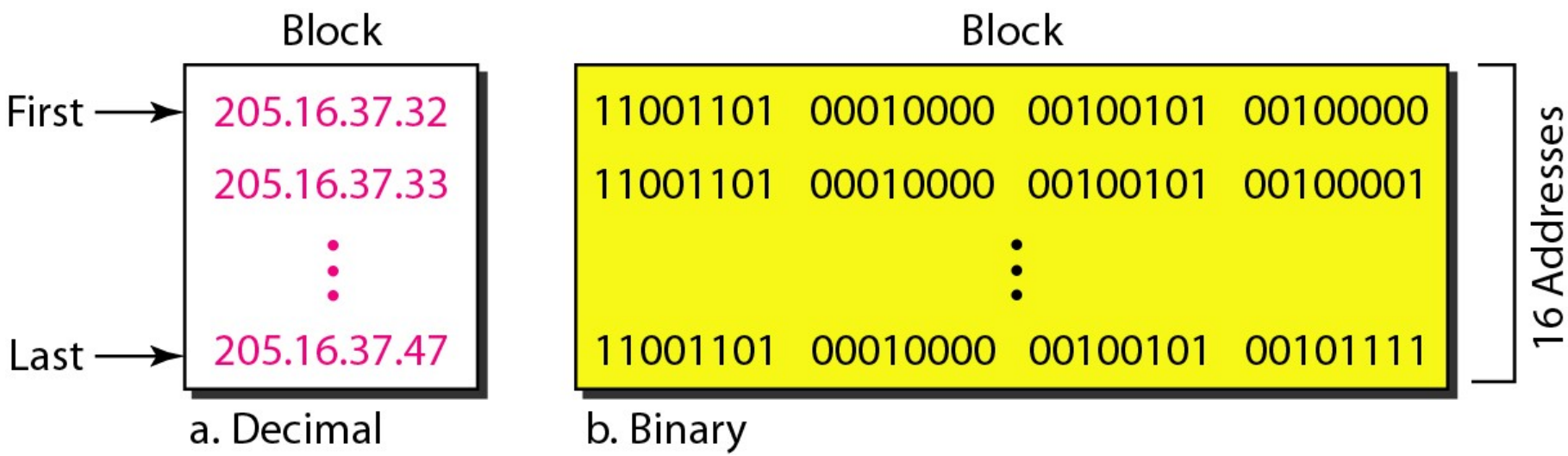
- If we set "32–28=4" ; rightmost 4 bits to 1, we get

  11001101 00010000 00100101 00101111

  IP In Dotted Decimal → 205.16.37.47

THE BLOCK

- The number of addresses in the block can be found by using the formula 2^{32-n} (where, **n** is CIDR Length).
- Here, the value of **n is 28** (e.g., IP Address - 205.16.37.39/**28**)
- Thus, number of addresses in the block where given IP address belongs are $2^{32-28} = 2^4 = 16$



SUBNETTING – 192.168.10.63/27

❖In which block does **192.168.10.63/27** lie?
❖Is **192.168.10.63/27** Network Address, Broadcast Address or Host/Usable IP?

- The number of addresses in the block can be found by using the formula 2^{32-n}
- Here, **n=27** → **Network Portion**
- 32 - n = 32 - **27** = 5 → **Host Portion**
- Total IP Addresses in a Block → $2^{32-n} = 2^{32-27} = 2^5 = 32$
  First IP is Network Address while Last IP is Broadcast Address
- Subnet Mask = **11111111.11111111.11111111.11100000** [255.255.255.224]
- Find No. of 1 bit in **Changing/Interesting Octet** → **k = 3**
- How many Blocks?
  - Total number of blocks (Subnetworks aka Subnets) = $2^k = 2^3 = 8$
- How many hosts in each block?
  - No. of hosts in each block = $2^5 = 32$, while **no. of usable hosts** = 32-2 =30

FINDING THE BLOCK – 192.168.10.63/27

- Changing Binary to last octet, as interesting octet is the last one,

  i.e., bit flip occurs after 27th bit, since **n=27**
  - 192.168.10.63 = **192.168.10.00111111**
- First IP in Block
  - 32-27=5, Thus, 5 zeros on the right = **192.168.10.00100000 = 192.168.10.32**
- Last IP in Block
  - 32-27=5, Thus, 5 ones on the right = **192.168.10.00111111 = 192.168.10.63**
- In which block does **192.168.10.63/27** lie?
  - Block is **"192.168.10.32 to 192.168.10.63"**
- Is **192.168.10.63/27** Network Address, Broadcast Address or Usable IP (Host IP)?
  - It's a broadcast address as it's the last IP in the block

SUBNETTING – 192.168.10.63/27

- How many hosts/IPs in each block? [Another Approach]:
- Subnet Mask (/27) = 11111111.11111111.11111111.11100000
- Changing octet: 11100000 (point when continuous 1 changed to zero)
- Interesting bit position

128	64	32	16	8	4	2	1
1	1	1	0	0	0	0	0

- Octet Summary Chart:

Powers of 2	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
No. of Hosts	**256**	128	64	32	16	8	4	2	1
No. of Blocks (Networks)	**1**	2	4	8	16	32	64	128	256
Subnet Mask	**0**	128	192	224	240	248	252	254	255
CIDR	*/24*	*/25*	*/26*	*/27*	*/28*	*/29*	*/30*	*/31*	*/32*

8 BLOCKS with 32 IPs EACH

Block 1	192.168.10.0 to 192.168.10.31
Network IP	192.168.10.0
Broadcast IP	192.168.10.31
Usable Range	192.168.10.1 to 192.168.1.30

Block 2	192.168.10.32 to 192.168.10.63
Network IP	192.168.10.32
Broadcast IP	192.168.10.63
Usable Range	192.168.10.33 to 192.168.10.62

Block 3	192.168.10.64 to 192.168.10.95
Network IP	192.168.10.64
Broadcast IP	192.168.10.95
Usable Range	192.168.10.65 to 192.168.10.94

Block 4	192.168.10.96 to 192.168.10.127
Network IP	192.168.10.96
Broadcast IP	192.168.10.126
Usable Range	192.168.10.97 to 192.168.1.126

Block 5	192.168.10.128 to 192.168.10.159
Network IP	192.168.10.128
Broadcast IP	192.168.10.159
Usable Range	192.168.10.129 to 192.168.10.158

Block 6	192.168.10.160 to 192.168.10.191
Network IP	192.168.10.160
Broadcast IP	192.168.10.191
Usable Range	192.168.10.161 to 192.168.10.190

Block 7	192.168.10.192 to 192.168.10.223
Network IP	192.168.10.192
Broadcast IP	192.168.10.223
Usable Range	192.168.10.193 to 192.168.10.222

Block 8	192.168.10.224 to 192.168.10.255
Network IP	192.168.10.224
Broadcast IP	192.168.10.255
Usable Range	192.168.10.225 to 192.168.10.254

Find Subnets with prefix **198.119.40.121/26**

1. What is Subnet Mask in dotted decimal format?
2. How many Subnets (Blocks) can be created?
3. List out the range of IP addresses in each subnet.
4. What is the first usable IP address of first block?
5. What is broadcast address of 2nd block?
6. What is network ID of 3rd block?
7. What is the usable IP range of last block?
8. What is the last IP Address of last block?
9. Is 198.119.40.121 host's IP, Broadcast IP or Network Address?

ANSWERS – 198.119.40.121/26

1. Subnet Mask – 255.255.255.192
2. 4 Blocks
3. Block wise range of IP Addresses
   - First Block – 198.119.40.0→198.119.40.63
   - Second Block – 198.119.40.64→198.119.40.127
   - Third Block – 198.119.40.128→198.119.40.191
   - Fourth Block – 198.119.40.192→198.119.40.255
4. First usable IP of 1st block is – 198.119.40.1
5. Broadcast address of 2nd block is – 198.119.40.127
6. Network ID of 3rd block is – 198.119.40.128
7. Usable IP range of last block is – 198.119.40.193→198.119.40.254
8. Last IP address of last block is – 198.119.40.255
9. 198.119.40.121 is host IP of 2nd block

OTHER EXAMPLES

- 220.117.44.121/25 → 2 blocks; 128 IPs in each block
- 220.117.44.121/26 → 4 blocks; 64 IPs in each block
- 220.117.44.121/27 → 8 blocks; 32 IPs in each block
- 220.117.44.121/28 → 16 blocks; 16 IPs in each block
- 220.117.44.121/29 → 32 blocks; 8 IPs in each block
- 220.117.44.121/30 → 64 blocks; 4 IPs in each block
- 220.117.44.121/31 → 128 blocks; 2 IPs in each block --- Not Used

Note:

- We express individual IP as **220.117.44.121/32**

SUBNET MASKS (CLASS B)

- 255.255.0.0 (/16)
- 255.255.128.0 (/17)
- 255.255.192.0 (/18)
- 255.255.224.0 (/19)
- 255.255.240.0 (/20)
- 255.255.248.0 (/21)
- 255.255.252.0 (/22)
- 255.255.254.0 (/23)
- 255.255.255.0 (/24)
- 255.255.255.128 (/25)
- 255.255.255.192 (/26)
- 255.255.255.224 (/27)
- 255.255.255.240 (/28)
- 255.255.255.248 (/29)
- 255.255.255.252 (/30)
- 255.255.255.254 (/31) – Not Used

172.16.191.250/18

- Subnet Mask – 255.255.11000000.00000000
- No. of blocks = 2^2 = 4 [Block start from 0, 64, 128, 192]
- No. of hosts in each block = 2^{14} = 16384
- Remember: 255.255.254.255 Plus 1 equals to 255.255.255.255
  [Adding 1 to 255 is not 256 but 1 is carried to number on the left octet]

Block 1	172.16.0.0 to 172.16.63.255
Network IP	172.16.0.0
Broadcast IP	172.16.63.255
Usable Range	172.16.0.1 to 172.16.63.254

Block 2	172.16.64.0 to 172.16.127.255
Network IP	172.16.64.0
Broadcast IP	172.16.127.255
Usable Range	172.16.64.1 to 172.16.127.254

Block 3	172.16.128.0 to 172.16.191.255
Network IP	172.16.128.0
Broadcast IP	172.16.191.255
Usable Range	172.16.128.1 to 172.16.191.254

Block 4	172.16.192.0 to 172.16.255.255
Network IP	172.16.192.0
Broadcast IP	172.16.255.255
Usable Range	172.16.192.1 to 172.16.255.254

SUBNET MASKS (CLASS A)

- 255.0.0.0 (/8)
- 255.128.0.0 (/9)
- 255.192.0.0 (/10)
- 255.224.0.0 (/11)
- 255.240.0.0 (/12)
- 255.248.0.0 (/13)
- 255.252.0.0 (/14)
- 255.254.0.0 (/15)
- 255.255.0.0 (/16)
- 255.255.128.0 (/17)
- 255.255.192.0 (/18)
- 255.255.224.0 (/19)
- 255.255.240.0 (/20)
- 255.255.248.0 (/21)
- 255.255.252.0 (/22)
- 255.255.254.0 (/23)
- 255.255.255.0 (/24)
- 255.255.255.128 (/25)
- 255.255.255.192 (/26)
- 255.255.255.224 (/27)
- 255.255.255.240 (/28)
- 255.255.255.248 (/29)
- 255.255.255.252 (/30)

10.255.255.252/10

- Subnet Mask – 255.11000000.00000000.00000000
- No. of blocks = 2^2 = 4 [Blocks start from 0, 64, 128, 192]
- No. of hosts in each block = 2^{22} = 4194304
- Remember: 255.254.255.255 Plus 1 equals to 255.255.255.255
  [Adding 1 to 255 is not 256 but 1 is carried to number on the left octet]

Block 1	10.0.0.0 to 10.63.255.255
Network IP	10.0.0.0
Broadcast IP	10.63.255.255
Usable Range	10.0.0.1 to 10.63.255.254

Block 2	10.64.0.0 to 10.127.255.255
Network IP	10.64.0.0
Broadcast IP	10.127.255.255
Usable Range	10.64.0.1 to 10.127.255.254

Block 3	10.128.0.0 to 10.191.255.255
Network IP	10.128.0.0
Broadcast IP	10.191.255.255
Usable Range	10.128.0.1 to 10.191.255.254

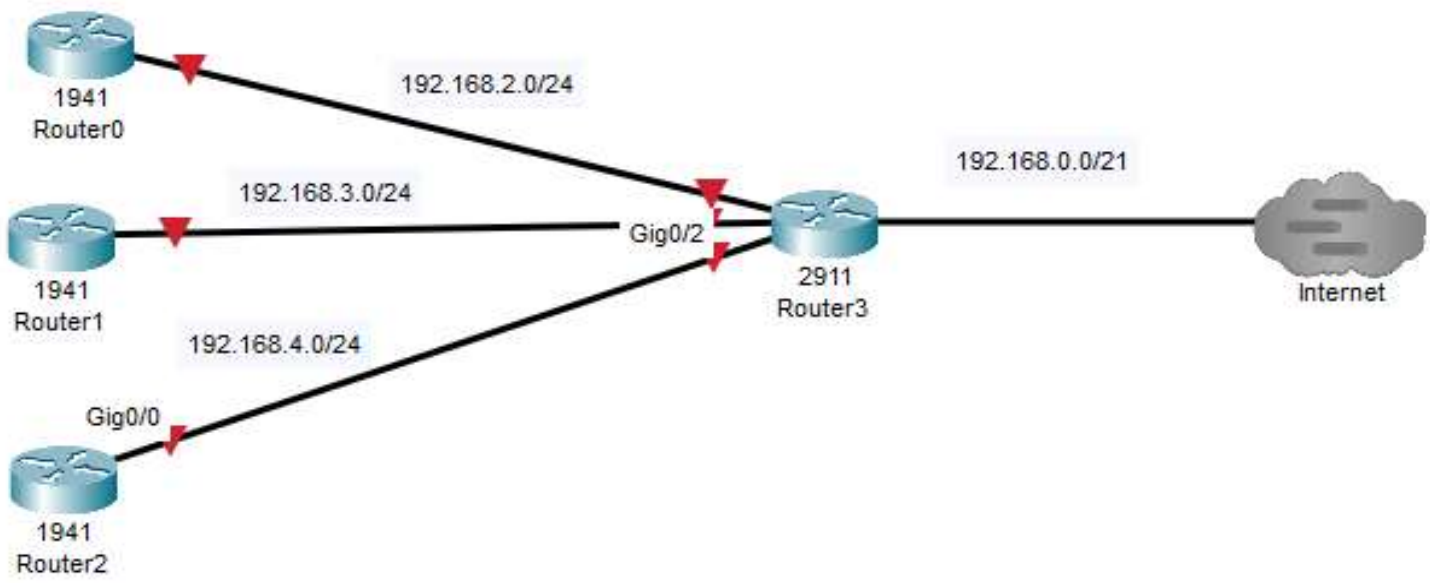
Block 4	10.192.0.0 to 10.255.255.255
Network IP	172.16.192.0
Broadcast IP	172.16.255.255
Usable Range	10.192.0.1 to 10.255.255.254

SUPERNETTING

- Taking the smaller networks and integrating them to make the larger one. **i.e., reverse of subnetting**
- Example: Three Networks each with Subnet Mask **255.255.255.0 [/24]**:
  - 192.168.2.0 – 11000000.10101000.00000010.00000000
  - 192.168.3.0 – 11000000.10101000.00000011.00000000
  - 192.168.4.0 – 11000000.10101000.00000100.00000000
- Find matching bits left to right until mismatch is got.
  - 192.168.2.0 – 11000000.10101000.00000010.00000000
  - 192.168.3.0 – 11000000.10101000.00000011.00000000
  - 192.168.4.0 – 11000000.10101000.00000100.00000000
- **New Network** [Setting all bits to 0 just from the mismatch]: **Supernet**
  - 11000000.10101000.00000000.00000000 [i.e. 192.168.0.0]
- **New Subnet Mask** [Setting all bits to 1 from start until the mismatch]: 11111111.11111111.11111000.00000000 [i.e., 255.255.240.0][/21]

BENEFITS OF SUPERNETTING

- Reduces the network traffic
  - Aggregated Router (e.g., Router3) can advertise the new network (192.168.0.0/21) thus no need to advertise the 3 previous networks.
- Minimizes the size of routing table



SUPERNETTING - EXAMPLE 2

Supernet the following networks into one single network:

- 202.168.197.0/24
- 202.168.198.0/24
- 202.168.199.0/24
- 202.168.200.0/24
- 202.168.204.0/24
- 202.168.206.0/24

SUPERNETTING - EXAMPLE 2

- 202.168.197.0/24 → 202.168.11000101.0
- 202.168.198.0/24 → 202.168.11000110.0
- 202.168.199.0/24 → 202.168.11000111.0
- 202.168.200.0/24 → 202.168.11001000.0
- 202.168.204.0/24 → 202.168.11001100.0
- 202.168.206.0/24 → 202.168.11001110.0

Network → 202.168.192.0 → 202.168.11000000.0

Subnet Mask → 255.255.240.0 → 255.255.11110000.0 [/20]

SUPERNETTING - EXAMPLE 2 - Binary

- 11001010.10101000.11000101.00000000
- 11001010.10101000.11000110.00000000
- 11001010.10101000.11000111.00000000
- 11001010.10101000.11001000.00000000
- 11001010.10101000.11001100.00000000
- 11001010.10101000.11001110.00000000

- **Network** - 11001010.10101000.11000000.00000000 - **172.168.192.0**
- **Subnet Mask-**11111111.11111111.11110000.00000000 - **255.255.240.0**
- **CIDR - /20**